17. Typeahead / Autocomplete

> **Interviewer:** Design the typeahead behind a search box — like Google's. As I type `"goo"`, you show me the top few completions — `"google"`, `"good morning"`, `"google maps"` — ranked by popularity, updating live as I type. This runs at Google scale: billions of queries a day. Go.

Candidate: Fun one — the interesting tension here is **sub-100ms responses on every single keystroke**, at a volume where you *cannot* rank anything live. Let me clarify the shape first.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Ranked by raw historical popularity, or does "trending right now" matter — e.g. a term that exploded in the last hour? 2. Is personalization in scope — my own search history nudging my suggestions — or just a global list for everyone? 3. Is a few hours of staleness acceptable, or do we need suggestions to reflect what people searched five minutes ago? 4. Locale/language — same trie for everyone, or per-language?
>
> **Interviewer:** Global popularity is the baseline; trending should surface within minutes-to-an-hour, not instantly. Personalization is a stretch goal, mention it at the end. A few hours of staleness for the *bulk* of the list is fine, but real spikes (breaking news) shouldn't take a day to show up. Assume per-locale.

Candidate: Then here's what I'm building:

Functional - `GET /suggest?q=<prefix>&locale=<x>` → top-K (say 5-10) completions, ranked by popularity. - Suggestions update from a live stream of queries — not a one-time static list. - Locale-aware: different trie per language/region.

Non-functional — and this is the whole game - **Latency:** sub-100ms, ideally sub-50ms, because it fires on *every keystroke*, not once per search. Anything slower feels broken, not just slow. - **Read volume is enormous** — every keystroke of every user, globally, billions/day. - **Ranking cannot happen at request time.** "Which of the millions of terms starting with 'goo' are most popular" is an aggregation over a huge corpus — computing that live, per keystroke, is a non-starter. - **Eventual consistency is fine** for popularity — trending can lag by minutes to an hour. - **Availability > completeness** — if the suggestion service is down, the search box must still work.

---

2. Estimation — why this is a memory/latency problem, not a storage problem

> **Candidate:** Let me size it, because the numbers tell me exactly where to spend effort.

```
Assume 6B searches/day globally (Google-scale ballpark).
  search QPS ~= 6,000,000,000 / 86,400 ~= ~70,000/sec (avg), peak ~3x ~= 210,000/sec

Typeahead fires on every KEYSTROKE, not every submitted search.
A 6-letter query like "google" -> ~6 keystrokes -> ~6 lookups per search.
  suggest QPS ~= 6 * search QPS(avg) ~= ~420,000/sec  (peak into the millions/sec)

That is the real number this design must survive: high hundreds of
thousands to low millions of reads/sec, each one needing an answer in <50ms.

Data size: keep, say, 50-100M distinct terms worth ranking per locale.
  each term ~50-100 bytes (text + score) -> 100M * 100B = ~10 GB per locale
  -> small enough to hold ENTIRELY IN MEMORY, per instance, per locale.

This one fact drives the whole design: it is a serving-from-RAM problem,
with a separate offline job that computes "what goes in RAM." The read
path must NEVER touch a database or do live ranking work.
```

---

3. V1 — the naive trie (no precomputation)

> **Interviewer:** Start simple. What's the most obvious data structure?

Candidate: A **trie** (prefix tree) — one node per character. Walk it letter by letter as the user types; when you reach the node for `"goo"`, that node's subtree contains every term starting with `"goo"`.

```
          (root)
            |
            g
            |
            o
            |
            o    <- node for prefix "goo"
         /  |  \
        g   d   s
        |
        l
        |
        e   <- "google" ends here
```

V1, naive: on each request, walk down to the `"goo"` node, then **DFS the entire subtree**, collect every completed word, count/rank them by popularity, and return the top-K.

Why it dies: the subtree under a common 2-3 letter prefix can have **millions** of terms. Doing a full subtree walk plus a sort, *per keystroke, per user, at 400k+ QPS*, is wildly too slow and CPU-expensive. This is the same lesson as every "rank on the fly at scale" problem: **you cannot afford to compute the answer live when the read rate is this high and the computation is this expensive.** We need to precompute it.

---

4. The key idea — precompute top-K at every node

> **Candidate:** So the fix: instead of ranking at *read* time, rank at *build* time. **Every node in the trie caches its own top-K list** — the K most popular full terms anywhere in that node's subtree. Building that list is expensive (it's basically an aggregation over the whole query corpus); reading it is just "walk N nodes, return a cached array." That's the trade the whole system is built around.

```
            (root)
              |
              g
              |
              o
              |
              o   <- node "goo", precomputed:
             /|\     topK = ["google", "good morning", "google maps",
            g d s             "good", "goodreads"]
            |
            l          GET /suggest?q=goo
            |            -> walk root -> g -> o -> o   (3 hops)
            e            -> read node.topK             (already sorted)
      "google"           -> return top 5-10            (O(1) after the walk)
```

Why precompute, say it plainly: ranking "top completions for every possible prefix" is an aggregation over **the entire query-frequency distribution** — you'd need to know, for every prefix, which of potentially millions of matching terms has the highest count. That is cheap to compute **once, in batch, over a bounded window of logs**, and prohibitively expensive to compute **per request, at hundreds of thousands of requests per second.** The read path becomes O(prefix length) — just a pointer walk — instead of O(subtree size).

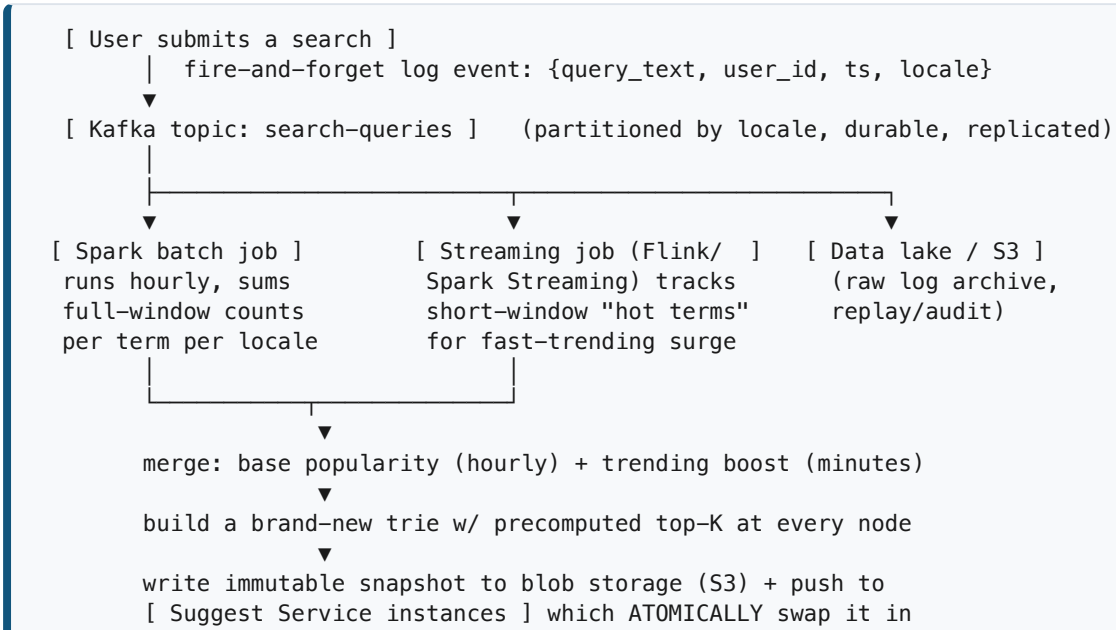
Building it: post-order traversal of the trie. Each leaf's "top-K of itself" is just itself. Each internal node's top-K is the **merge** of its children's top-K lists, sorted by count, truncated to K. Since K is small (say 10) and each child contributes at most K candidates, merging is cheap — this is why the precompute step, while heavy in aggregate, is tractable to run as a batch job.

---

5. Where does "popularity" come from? The build path (offline aggregation)

> **Interviewer:** Where do these counts even come from — and how do they stay current as new terms trend?

Candidate: This is the second half of the system, and it's fully decoupled from serving. Every completed search is logged; a pipeline aggregates those logs into counts and rebuilds the trie periodically.



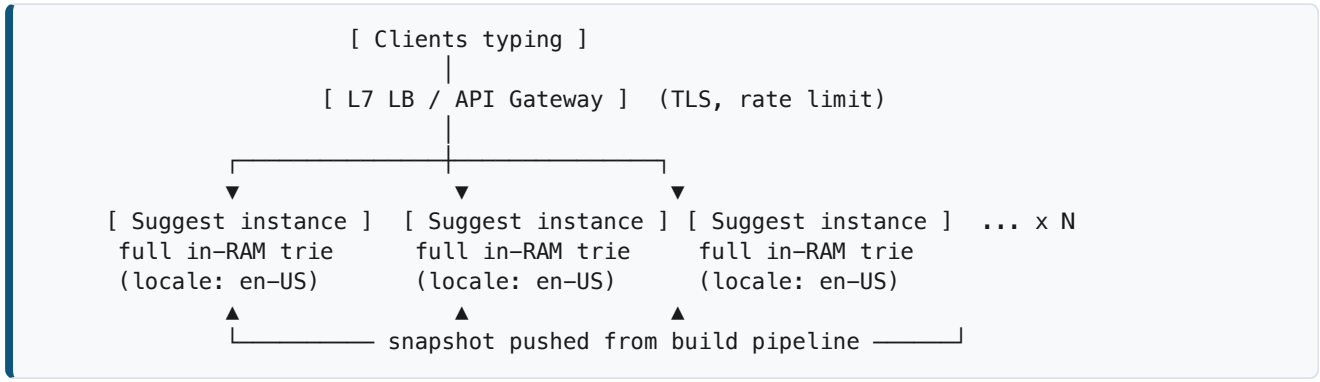
- **Kafka** decouples "a search happened" from "the suggestion trie gets updated." Search submission is never blocked by, or coupled to, the aggregation pipeline.
- **Spark batch job** (e.g. hourly) computes the durable, high-confidence popularity ranking over a large window — this is the bulk of the top-K list and it's fine for it to be a bit stale.
- **A lightweight streaming job** (shorter window, e.g. 1-5 min) tracks sudden spikes — a breaking-news term — and its output is merged on top of the batch result so trending shows up in minutes, not the next hourly cycle, without redoing the whole expensive rebuild.
- **The rebuilt trie is versioned and swapped in atomically** — readers never see a half-built structure; old version simply gets discarded once the new one is live.

---

6. Scaling reads — serving 400k+/sec from memory

> **Interviewer:** Good. Now — 400,000+ reads a second, sub-50ms each. How do you actually serve that?

Candidate: Every serving instance holds a **full, read-only copy of the trie in memory** — no network hop to a database or even to a separate cache tier on the hot path. Since the whole dataset for a locale is only ~10GB (Section 2), it comfortably fits in one instance's RAM. Scaling reads is then just "add more identical stateless instances behind a load balancer" — completely horizontal, no shared bottleneck.



- **No cache-miss path to a DB.** There is no "if not in cache, go read Postgres" — the entire dataset is always resident. This is what makes p99 sub-50ms achievable: it's a pointer walk plus an array copy, nothing else.
- **Hot prefixes are naturally handled.** Everyone types common letters first ("a", "the", "goo…") — those are exactly the nodes every replica already has fully cached. There's no "hot key" problem here the way there is in a counter-based design, because there's no shared mutable state on the read path at all — every instance is independently self-sufficient.

---

7. Sharding the trie — when one locale's dataset gets too big

> **Interviewer:** Fine for 10GB. What if one locale (say English) balloons to 500GB of terms because you're tracking every long-tail multi-word query?

Candidate: Then it stops fitting in one box's RAM and I need to **shard the trie**, by **first character(s) of the prefix**.

```
Shard by first 1–2 letters of the prefix:

Shard 0: a*, b*      Shard 1: c*, d*, e*     Shard 2: f*, g*, h*   ...
[instances]           [instances]              [instances]

GET /suggest?q=goo
   -> route by first letter 'g' -> Shard 2
   -> Shard 2 instances hold full "g*" subtree in RAM, walk to "goo",
      return cached topK
```

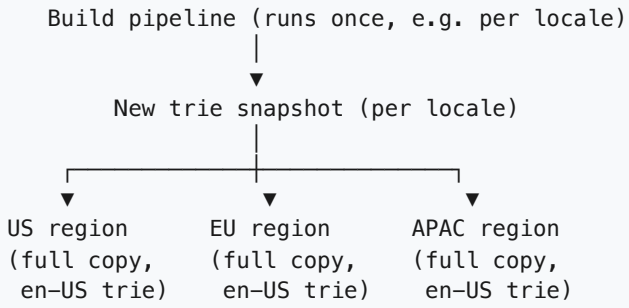
- A **routing layer** in front (or client-side hashing at the gateway) sends the request to the shard owning that prefix's first character(s), then it's the same in-memory walk as before, just over a smaller subtree.
- Choose the split point (1 letter vs 2) so shards land roughly balanced — letter frequency isn't uniform (`s`, `c`, `a` are far more common starting letters than `q`, `x`, `z`), so in practice this is a **weighted** split by observed volume, not a naive alphabetical 26-way split.
- Each shard is *itself* replicated across many instances for read throughput, same as Section 6 — sharding solves the **size** problem, replication solves the **QPS** problem. They compose.

---

8. Multi-region

> **Interviewer:** This is a global product. Someone in Tokyo shouldn't round-trip to a US data center on every keystroke.

Candidate: Since staleness is acceptable, this is the easy kind of multi-region problem — **copy, don't coordinate.**



Locale and region are independent axes: a region might host multiple locale tries (e.g. the US region serving `en-US` and `es-US`); the routing layer picks the trie by the request's `locale` param, and geo-routing picks the *region*. Because we already accept minutes of staleness, we don't need synchronous cross-region replication — each region just picks up the next snapshot push on its own schedule.

---

9. Latency — where the milliseconds actually go

> **Interviewer:** You keep saying sub-50ms. Walk me through the budget.

Candidate: Breaking down a single request:

```
Client keystroke
  -> (debounce ~100-150ms client-side, doesn't count against server latency,
      but cuts request volume hugely — a 6-letter word becomes ~1-2 requests
      instead of 6)
  -> network to nearest edge/region:            ~5-15ms (geo-routing keeps this low)
  -> LB + gateway (TLS, rate limit):             ~1-2ms
  -> Suggest instance: walk trie (few hops)
       + copy top-K array:                       <1ms  (pure in-memory pointer chase)
  -> response back over network:                 ~5-15ms
  ------------------------------------------------------------------
  total, same-region:                            ~15-35ms  comfortably under budget
```

- The **only** place real "work" happens is the trie walk, and it's deliberately O(prefix length), not O(data size) — that's the entire point of precomputing top-K at every node.

- **Client-side debouncing** is a cheap, free win: waiting ~100ms after the last keystroke before firing cuts request volume by roughly the average word length, directly reducing load without any backend change.
- If we ever need to shave more: **keep top-K lists denormalized as flat, contiguous arrays** (not lists of pointers to scattered objects) so a "hit" is one memory read, not several cache-unfriendly pointer chases.

---

10. Consistency — what's strong, what's eventual (and why that's fine)

> **Interviewer:** Two users type "goo" a second apart and get slightly different top-5. Bug?

Candidate: Not a bug — expected, and desirable to keep the system simple.

- **Strong/consistent within a single served snapshot.** Every request hitting a given instance sees the *same* trie until the next atomic swap — no partial or half-updated reads.
- **Eventual across instances/regions.** Different instances may be on slightly different snapshot versions for a short window while a new one rolls out — one user might see `"google"` ranked #1 and another (on a not-yet-updated replica) sees it at #2. Nobody notices or cares; this is the textbook case for eventual consistency, because the "correct" answer is a fuzzy popularity ranking, not a financial balance.
- **Eventual, with a bounded staleness SLA.** The batch rebuild (hourly) plus the streaming trending layer (minutes) bounds *how* stale things get. We explicitly trade freshness for simplicity and read-path speed — computing "live, exact, globally-consistent" popularity would require the kind of expensive real-time global aggregation we ruled out in Section 4.

---

11. Async — the whole aggregation pipeline lives off the critical path

> **Interviewer:** What happens if the build pipeline breaks at 2am?

Candidate: Nothing happens to users — that's the design goal. Logging a search and serving a suggestion are **fully decoupled** from rebuilding the trie:

```
Search submission  ⟶ Kafka (async, fire-and-forget)  ⟶ never blocks the user
Suggest request    ⟶ reads ONLY the last-good snapshot ⟶ never touches the build path
```

If the Spark job fails mid-run, or the streaming trending job falls behind, the **old snapshot just keeps serving.** We retry the build next cycle. The read path has zero awareness that anything went wrong upstream — it's reading a static, versioned artifact. This asymmetry (cheap synchronous reads, expensive async writes-of-the-index) is the load- bearing idea of the whole design.

---

12. Caching — and the DB↔cache-equivalent data flow

> **Candidate:** To be precise about roles, since it's easy to conflate "trie" and "cache" here: the in-memory trie **is** the cache — there is no separate "real" database that it's caching a copy of. The closest thing to a "database" is the append-only Kafka log / data lake of raw search events, and we deliberately **never** query that on the read path.

```
QUERY PATH (must be <50ms):
  user types "goo" -> Suggest Service -> walk in-memory trie -> return topK
  (zero disk I/O, zero DB call, zero live computation)

BUILD PATH (offline, hourly + streaming trend layer):
  raw search logs (Kafka) -> Spark/Flink aggregation -> new trie w/
  precomputed topK at every node -> push snapshot -> atomic swap
```

Invalidation is simply "the old snapshot is discarded once the new one is live" — there's no per-key invalidation logic because we never mutate the served structure; we only ever **replace the whole thing, atomically, in one shot.** That sidesteps an entire category of cache-consistency bugs (partial invalidation, stale-key races) that a mutable cache would have to deal with.

---

13. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Serving structure (read path)	**In-memory trie with precomputed top-K per node**, full copy per instance	O(prefix length) lookup, no network hop, no live ranking. *Not a live DB query per keystroke* — ranking millions of matching terms per request at 400k+ QPS is computationally infeasible and would blow the latency budget by orders of magnitude.
Alternative serving shape	Flat KV: `prefix -> topK` in Redis	Simpler to reason about/shard (just a hash lookup), at the cost of storing every prefix explicitly instead of sharing trie structure. Valid alternative; trie wins when you want structural sharing and easy prefix-range logic; flat KV wins if you want off-the-shelf Redis ops and simpler sharding.
Raw event ingestion	**Kafka**, partitioned by locale	Durable, ordered-per-partition, replayable log; the aggregation job reads from it independently of request traffic — logging a search never blocks or slows the searcher. *Not synchronous writes to the aggregation store* — that would couple search latency to indexing work.
Batch aggregation (bulk of the ranking)	**Spark**, hourly	Cheaply computes exact counts over a huge window of logs; batch is the right tool because the output only needs to be as fresh as "once an hour," and batch throughput per dollar beats streaming for bulk recompute.
Streaming aggregation (trending)	**Flink / Spark Streaming**, short window (minutes)	Surfaces breaking-news-style spikes fast, merged on top of the batch ranking, without re-running the expensive full rebuild every few minutes. *Not relying on batch alone* — an hourly-only

Concern	Choice	Why this, and why not the alternative
		cycle would make genuinely trending terms invisible for up to an hour, which the interviewer explicitly said wasn't acceptable.
Durable raw log / archive	**Data lake (S3 / HDFS)**	Cheap, durable, replayable storage for the full history of search events — needed for reprocessing, backfills, and auditing the popularity pipeline, but never on the serving hot path.
Why not query the DB live	—	There **is** no live "database" serving suggestions — that's the point. A traditional DB (SQL or NoSQL) is great at durable storage and flexible queries, but even a fast KV round-trip (~1-5ms network) eats meaningfully into a <50ms budget fired on every keystroke, and computing "top-K starting with this prefix, ranked by popularity" as a live query is exactly the expensive aggregation we push offline.
Snapshot distribution	**Push a versioned, immutable artifact to every instance/region; atomic pointer swap**	Readers never see a half-built structure; rollout/rollback is just "point at a different immutable version." *Not in-place mutation of the live trie* — concurrent readers walking a structure being edited underneath them is a correctness nightmare.
Load balancing	**L7 / API Gateway**	Service is small, stateless, read-only — routing is simple; L7 earns its keep for TLS termination and per-client rate limiting so one abusive client can't fire uncapped requests on every keystroke.

Say-it-out-loud justification: "*The read path is nothing but a walk down an in-memory trie to a node that already has its top-K precomputed, so it's a few memory hops with zero live ranking work and zero database calls; all the expensive work — counting billions of queries and deciding what's popular — happens offline in a Spark batch job plus a fast streaming layer for trending, and the result is pushed out as an immutable, versioned snapshot that gets swapped in atomically. That split is what makes sub-50ms possible at this volume — you simply cannot rank live at this QPS, so you precompute and just serve.*"

---

14. Failure modes & graceful degradation

Failure	What happens	Mitigation
One Suggest instance crashes	Its in-memory trie is gone with it	Stateless, replicated instances — LB routes around it; new instance boots and pulls the latest snapshot
Whole cache/serving tier degraded	Risk of falling back to nothing	Fall back to a small global "top-1000, no personalization, no locale" static set kept everywhere, rather than fail the request entirely
Batch build job fails	New popularity data delayed	Old snapshot keeps serving; job retries next cycle — stale is acceptable, broken is not
Streaming trend job falls behind	Breaking-news terms surface a bit later	Falls back to whatever the last hourly batch had; degraded freshness, not degraded correctness
Kafka/log pipeline down	No new events aggregated	Serving is entirely unaffected (reads never touch Kafka); once recovered, backlog is drained and next rebuild catches up

Failure	What happens	Mitigation
Suggest Service fully down	No suggestions	Client-side: search box itself still works, just without the dropdown — suggestions are a nice-to-have layered on search, never a hard dependency

---

15. Follow-up curveballs

> **Interviewer:** How would you personalize this per user without blowing the latency budget?

Candidate: Keep the global top-K as the base, and layer a **thin, small per-user signal** on top rather than personalizing the whole trie (which would be enormous per-user state). Concretely: maintain a short list of the user's own recent searches (client-side or in a tiny per-user cache), and merge/re-rank a few of those into the global top-K at serving time — a cheap O(K) merge, not a separate massive personalized index. Personalization never touches the shared trie build.

> **Interviewer:** How do you handle typos — "gogle" should still suggest "google"?

Candidate: That's a separate concern from exact-prefix matching — I'd add a **fuzzy layer in front of (or alongside) the exact trie**: if the exact-prefix lookup returns nothing or very few results, fall back to an edit-distance-tolerant index (or a phonetic/n-gram index) for a secondary suggestion pass. Keep it decoupled so the common case (exact prefix match, the vast majority of keystrokes) stays on the cheap fast path, and only the miss case pays for fuzzy matching.

> **Interviewer:** A single term suddenly spikes 100x in the last 5 minutes — a breaking news event. How fast does that reach users, end to end?

Candidate: Within the streaming layer's window — say a few minutes. The events land in Kafka, the streaming aggregator (short window) picks up the spike, recomputes just the affected prefixes' trending boost, and that gets merged on top of the current base snapshot and pushed out. We wouldn't rerun the full hourly batch rebuild for this — the streaming layer's whole reason to exist is to shortcut that latency for exactly this scenario.

> **Interviewer:** What if two locales share almost the same term set (e.g. `en-US` and `en-GB`) — are you duplicating everything?

Candidate: Some duplication is fine and cheap (10GB-ish per locale, memory is not the bottleneck), but if it becomes wasteful, the build pipeline can compute one **base** trie for the shared language and layer small locale-specific overrides/boosts (e.g. `"footy"` ranks higher in `en-GB`) on top, rather than two fully independent tries. That's a build-path optimization, though — it doesn't change anything about the serving path.

---

16. Deep-dive: "Why not just run a `LIKE 'prefix%'` query against a database?"

> **Interviewer:** This whole trie-plus-offline-pipeline machine feels heavy. Why not keep it simple: one table `search_terms(text, count)`, a B-tree index on `text`, and on every keystroke run `SELECT text, count FROM search_terms WHERE text LIKE 'goo%' ORDER BY count DESC LIMIT 10`? Or even simpler — aggregate counts live from the raw logs when asked. Why is that not good enough?

Candidate: It's the right instinct to reach for first, and it's worth separating the two things people usually bundle together: **finding rows that match the prefix** vs. **ranking those rows by popularity**. The first part is actually fine at moderate scale; the second part is where it falls apart, and the QPS in Section 2 is what makes it fatal here.

Prefix matching alone isn't the killer

A B-tree index on `text` *can* serve `LIKE 'goo%'` efficiently — a leading-wildcard-free prefix is just a range scan (`text >= 'goo' AND text < 'gop'`), so the database doesn't need to scan the whole table to find candidates. If that were the whole story, a DB might limp along.

Ranking by popularity is the part that breaks

The index is sorted by `text` (alphabetically), **not by `count`**. So `ORDER BY count DESC LIMIT 10` on top of a prefix range scan can't be satisfied by walking the index in order — the engine has to **materialize every matching row, then sort by count, then take the top 10**. For a common 2-3 letter prefix like `"goo"` or `"a"`, that "every matching row" set can be **hundreds of thousands to millions of rows**, and it's redone **from scratch on every single keystroke, for every user, worldwide.** A live aggregation over raw log events is strictly worse — now you're not even scanning pre-aggregated counts, you're re-summing raw events on the hot path.

```
GET /suggest?q=goo   (fires again ~400k–1M+ times/sec globally, per Section 2)
       │
       ▼
SELECT text, count FROM search_terms
  WHERE text LIKE 'goo%'      <- index range scan, cheap-ish: finds ~2M matching rows
  ORDER BY count DESC          <- CANNOT use the text-index for this; must sort what
  LIMIT 10                        it just found -> materialize + sort ~2M rows, EVERY TIME
       │
       ▼
...repeat this, per keystroke, at hundreds of thousands of times per second,
against ONE logical dataset -> connection pools saturate, sort buffers thrash,
replicas fall over, and it's still redoing IDENTICAL work a million times a
second because "goo" didn't change between two requests a millisecond apart.
```

The tell: two different users typing `"goo"` a second apart get **the exact same answer**, yet the naive design recomputes it from scratch both times. That's the waste a precomputed structure eliminates — compute once (offline), serve forever (until the next rebuild).

Concern	DB `LIKE 'prefix%' ORDER BY count` (or live aggregation)	In-memory precomputed top-K trie
Per-request cost	O(matching rows) materialize + sort — can be millions of rows for short prefixes, *every request*	O(prefix length) pointer walk + O(K) array copy — independent of corpus size
Sustaining 400k-1M+ QPS	Requires a fleet of read replicas, connection pooling, and still hits sort/CPU limits under identical repeated queries	Trivial — stateless replicas, no shared bottleneck, no query execution at all
Latency at p99	Index scan is fast; the sort of large match sets is not — tail latency blows past 100ms as prefixes get shorter/more common	Sub-millisecond in-memory walk; latency is flat regardless of how popular the prefix is
Redundant work	Recomputes the identical ranking for the identical prefix on every single request, even a millisecond apart	Computed once per rebuild cycle; reads just replay the cached answer
Where ranking happens	At request time, under the exact load spike you're trying to serve	Offline, in batch, decoupled from request volume entirely (Section 5)
Freshness	Technically "live" — reflects `count` the instant it's updated	Bounded-stale (minutes to an hour) — an explicit, accepted tradeoff (Section 10)
Operational shape under spike	Gets *worse* exactly when traffic is highest — the failure mode is coupled to success	Serving cost is flat; only the offline build job scales with corpus size, not request rate

> **Rule of thumb:** if the same read (same prefix) is going to be asked by *millions of different callers* before the underlying data meaningfully changes, that's the signature of a **precompute-once, serve-many** problem — doing the expensive ranking work live, per request, is paying for the same answer over and over. A database is the right tool when each query's answer is genuinely unique to that request; it's the wrong tool when a whole class of requests (every prefix) shares one slowly-changing answer.

---

17. Deep-dive: keeping the trie fresh under a trending spike, at billions-of-terms scale

> **Interviewer:** Two things at once. First: you said trending should surface within minutes, but you also said the trie is rebuilt and swapped **wholesale**. Walk me through exactly what breaks if I try to mutate the *live* trie in place instead of rebuilding it, at the moment a term is going viral. Second: what if the term corpus isn't 100M — it's **billions** of long-tail multi-word queries across every locale? Does "10GB, fits in RAM" still hold?

Candidate: These two are actually the same underlying tension — **freshness vs. a structure that's simultaneously being read by hundreds of thousands of concurrent lookups** — just triggered by

different pressures (a hot term vs. raw size). Let me take them in order.

Why you can't just mutate the live trie in place

Say `"tacos"` explodes 200x in the last 90 seconds (a viral food trend). The naive fix: "just update the `topK` array on the relevant nodes right now." Here's exactly what goes wrong:

```
Reader thread (mid-request, walking to node "t"->"a"->"c"):
  1. dereference node("tac").topK pointer
  2. read topK[0..K-1] to build the response array
                                      ▲
                                      │  <-- Writer thread, concurrently, RIGHT HERE:
                                      │      recomputing node("tac").topK because "tacos"
                                      │      just re-entered the top-5 for that prefix
                                      │
                                      └── writer is halfway through overwriting the array:
                                          topK = ["taco bell", "tacos", <garbage/uninitialized>,
                                                  "tacoma", "tack"]

Reader now returns a TORN READ: part old ranking, part new ranking, possibly a
slot that's neither -> at best a flickering, inconsistent suggestion list;
at worst a crash if the update also touches child pointers (subtree structure)
while a walk is mid-traversal through them.
```

You could guard every node with a lock, but at hundreds of thousands of reads/sec *per popular node* (`"g"`, `"a"`, `"the"` are read constantly), a per-node lock turns the hottest, most-read part of the trie into the exact serialized-hot-key bottleneck this whole design exists to avoid. **Mutating shared, concurrently-read state in place is the anti-pattern** — which is exactly why the design builds a whole *new* structure and swaps a single pointer (Section 5), rather than editing the live one. The question is how to make that swap happen fast enough for a spike to show up in minutes, and cheap enough to not mean re-deriving the *entire* corpus every time.

The offline pipeline's lag, made concrete

```
T+0:00   "tacos" query volume starts spiking (viral moment begins)
T+0:00 → T+2:00   events flow into Kafka, partitioned by locale — no lag added here,
                  this is just normal ingestion
T+2:00   streaming aggregator's window closes (a 2-minute tumbling window) and it
         finally HAS ENOUGH DATA to notice "tacos" is trending, not just noisy
T+2:05   streaming job emits an updated trending-boost for the "tac..." prefixes
T+2:05 → T+2:20   merge step combines trending boost with current base ranking,
                  rebuilds/patches just the affected nodes, produces a new artifact
T+2:20 → T+2:40   push snapshot to instances/regions; each instance atomically
                  swaps its pointer on receipt (not synchronized across instances)
-------------------------------------------------------------------------
total time-to-visible, worst case: ~2.5-3 minutes  <- matches the "minutes, not
                                                      instantly" requirement from
                                                      Section 1, but ONLY if every
                                                      stage above stays this fast
```

If the merge/rebuild step is written as "recompute the *entire* trie from scratch" rather than "patch just the nodes on the path to `tacos`," this lag balloons — a full rebuild over billions of terms can take much longer than 15 minutes, and now trending is bottlenecked on total corpus size, not on the size of what actually changed.

The billions-of-terms sizing problem, and a shard hotspot

Section 2's "~10GB, fits in one box" assumed ~100M terms. Push that to billions of long-tail multi-word queries and the math changes:

```
2B terms * ~100 bytes/term (text + count + topK contribution) ≈ 200 GB per locale
  -> does NOT fit in one instance's RAM anymore -> MUST shard (Section 7)

Shard by first letter, unweighted:
  Shard "g*": g, go, goo, goodreads, google, google maps, good morning, ...
     -> "g" and "s" are extremely common prefix letters in English
     -> Shard "g*" alone might hold 15% of ALL terms and take 15% of ALL traffic

  Shard "x*", "q*", "z*": tiny fraction of terms, tiny fraction of traffic

 Result if shards are split naively (26-way, one letter each):
    Shard g: ████████████████  (hot: oversized AND overloaded)
    Shard s: ████████████      (hot)
    Shard q: |                 (idle, wasted capacity)
    Shard x: |                 (idle, wasted capacity)

 Now ADD a viral spike on top: "tacos" lives in Shard "t". If "t" was already a
 medium shard, a 200x spike in queries for that ONE prefix can push Shard t's
 QPS past what its replica count was provisioned for -> that shard's p99 latency
 degrades while every other shard sits comfortably under load, because
 replica counts were sized for AVERAGE letter frequency, not for a live spike
 landing on one shard.
```

This is the trie-sharding analog of the burger-giveaway's stranded-bucket problem: **static, uniform partitioning doesn't match non-uniform, time-varying demand.**

Ranked fixes

1. Offline batch rebuild + atomic swap (the baseline, already in place) — simple, correct, zero risk of torn reads. But on its own it doesn't solve billions-of-terms sizing (still one artifact per shard) and, if implemented as "rebuild everything," it's too slow to hit a minutes-level trending SLA at this size.

2. Incremental/delta updates instead of full rebuilds — rather than recomputing the whole trie, the streaming layer computes a small **delta**: just the `topK` for the handful of nodes on the path to the spiking term(s). Apply deltas as an immutable **overlay** — a small map of `{node-path -> patched topK}` consulted first, falling back to the base snapshot — rather than editing base-trie nodes in place. The overlay itself is swapped in atomically (same trick, much smaller object), so reads are still torn-read-free, but the *cost* of publishing an update is now proportional to what changed, not to corpus size. This is what makes "minutes" achievable even at billions of terms.

3. Shard the trie by prefix, but rebalance by observed load, not alphabet — instead of a fixed 26-way split, size shard boundaries (and replica counts *per shard*) using measured query volume — a hot letter like `"s"` gets more, narrower shards and more replicas; a cold letter like `"q"` gets folded in with neighbors. Additionally, autoscale replica count **per shard** independently in response to real-time QPS, so a viral spike on Shard `t` provisions more replicas of *that* shard specifically, instead of over-provisioning every shard uniformly just to survive worst case.

4. Serve stale-but-fast as the default posture under any pipeline hiccup — whatever else is true, a reader never blocks on the build/merge/rebalance machinery. If the delta overlay or a rebalance is

delayed, instances keep answering from the last-good base snapshot. Users get a slightly-behind ranking, never a slow or failed one — staleness is the release valve for every failure mode above.

Recommended approach: run all four together, because they solve different axes of the same problem — #1 (hourly full rebuild) keeps the base ranking correct and simple; #2 (delta/overlay) is what actually delivers minutes-level trending without paying for a full rebuild; #3 (load-aware sharding + per-shard autoscaling) is what keeps a single viral prefix from overwhelming one shard as the corpus grows past one box's RAM; and #4 (always serve the last-good state) is the safety net that makes every one of the above allowed to be "eventually successful" instead of needing to be instantaneous and infallible. None of them requires ever mutating a structure that's being concurrently read.

> **What to say out loud:** *"I'd never mutate the live trie in place — concurrent readers walking it while it's being edited is a torn-read/crash risk, so every update is still an atomic swap, just of something smaller than the whole trie: a small delta overlay for the nodes that actually changed, rebuilt in a couple minutes by a streaming job, layered on top of an hourly full rebuild for the base ranking. For size, once one locale's corpus outgrows a box's RAM I shard by prefix like before, but I size shard boundaries and replica counts by observed query volume rather than alphabet, and autoscale replicas per shard so a viral spike on one prefix doesn't degrade shards that aren't under load. And whatever's slow or broken upstream, the read path always has a last-good snapshot to fall back to — staleness, never unavailability."*

Summary — the mental model

Typeahead is **"answer in under 50ms, on every keystroke, at hundreds of thousands to millions of requests per second, with a ranking that would be far too expensive to compute live."** The winning idea: **push all the expensive ranking work offline** — aggregate the query stream in Kafka, batch-rank with Spark (plus a fast streaming layer for trending), and precompute the top-K completions **at every node of a trie** so a lookup is just a pointer walk. Serve that trie **entirely from memory**, replicated across instances and regions, shard it by prefix when it outgrows one box, and swap in new versions atomically. Consistency is eventual and bounded-stale on purpose — the read path never touches raw logs, a live database, or a live ranking computation, because at this scale "compute on read" and "sub-50ms" are mutually exclusive.